EQUAMMERCE: An Accessibility Layer for E-Commerce

Voice-first, multilingual shopping on real Indian e-commerce sites for people with disabilities

Megathon 2026 · Bhashini track · Team 27

1 Solution

Online shops are built for people who can see small text, aim a mouse, type quickly and read English. Screen readers alone do not fix this: a product page on Amazon.in has about 200 controls, review pages run to hundreds of opinions, and checkout hides costs in fine print. Equammerce is a browser extension (Chrome or Firefox on desktop or Android) with a FastAPI backend. It works *on the real Amazon.in and Flipkart pages*, in the user's own logged-in tab, and lets a person **talk, tap or press one switch** to **search, understand a product, compare, add to cart, check out and manage orders**, in **English, Hindi, Telugu and Malayalam**. Every answer is spoken through Bhashini and shown as large, structured text; every fact comes from the page; every consequential step waits for the user's "yes"; and payment is always entered by the user.

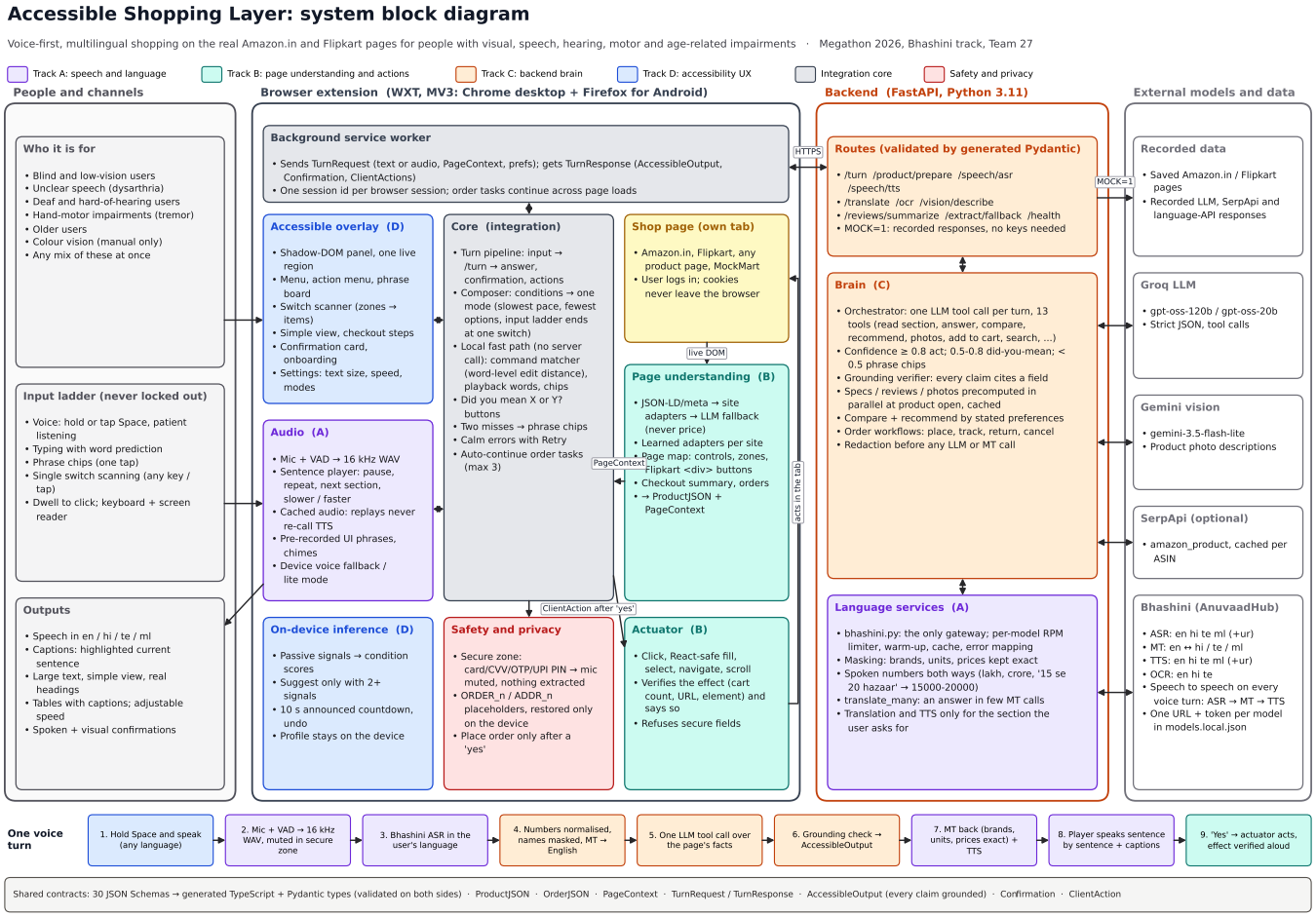


Figure 1. System block diagram. The extension reads the live page and acts in the user's tab; the backend never sees cookies and never talks to the shop. Colours mark the four work tracks; the strip shows one speech-to-speech voice turn.

2 Approach

Contracts first. 30 JSON Schemas (ProductJSON, PageContext, TurnRequest/Response, AccessibleOutput, Confirmation, ClientAction, ...) generate TypeScript and Pydantic types, and every endpoint validates input and output. Four tracks built against recorded mocks (`MOCK=1`) in parallel and integrated on `main` behind `make check`.

Page understanding (extension). A three-tier extractor reads the live page fastest-first: JSON-LD/meta tags → site adapters for Amazon.in and Flipkart → an LLM fallback on redacted visible text, which may fill missing fields but *never the price*. Fields the fallback finds are turned into **learned selectors** for that site, so the next page needs no LLM call. A **page map** lists every control with a stable id, label and zone (search, results, product, cart, checkout, orders); on Flipkart it also finds the `<div>` buttons that React Native Web renders without any button role. Checkout totals, delivery PIN and payment method are read from visible labels; order numbers and addresses leave the device only as `ORDER_n`/`ADDR_n`. An actuator clicks, fills (React-safe), selects and navigates, then **verifies the effect** (cart count, URL) and says whether it worked.

Backend brain. Each `/turn` makes **one LLM tool call** (Groq gpt-oss) over the page's facts and picks one of 13 tools (read section, answer, compare, recommend, describe photo, add to cart, search, order task, ...). Confidence ≥ 0.8 acts, 0.5–0.8 asks "Did you mean X or Y?", below 0.5 offers phrase chips. When a product opens, the specs, reviews and photo summaries are written **in parallel** and cached; a **grounding verifier** checks that every claim cites a ProductJSON field and falls back to templates if any claim fails. Comparison ranks by the user's stated preferences, never by their disability profile. Place, track, return and cancel are **step machines** with read-back confirmations.

Language (Bhashini). A single gateway calls AnuvaadHub ASR, MT, TTS and OCR models with a per-model rate limiter, start-up warm-up, caching and calm error mapping. Brand names, units and prices are **masked** before translation and restored after; spoken numbers are normalised both ways ("pandrah se bees hazaar" → 15000–20000; prices read as words in the listener's language). Only the section the user asks for is translated and spoken.

Accessibility layer (extension). An overlay in a shadow DOM with one live region renders every answer as speech and as large structured text. A rule-based **composer** merges any combination of conditions into one mode (slowest speech, fewest options, an input ladder that always ends at one switch). On-device inference suggests a mode only when two independent signals agree, with an announced 10-second countdown and undo. The profile never leaves the device.

3 Features

Shopping scenarios, end to end by voice, keys, chips or one switch

Browse and filter by voice	"Search for earphones under 2000": the real tab goes to the shop's search URL; results are read from the page; budget, rating and priorities filter and rank them.
Specs as a short summary	Key specs as a spoken list and a captioned table, with units kept exact; "the page doesn't list this" when a field is missing.
Reviews → pros, cons, verdict	Good points and drawbacks with mention counts, built from the shop's own "Customers say" and the reviews actually read, stating what it is based on.
Photos and size charts in words	Gemini describes the main photo (more on request): colour, shape, what's in the box; size-chart images among the product photos are described the same way. Printed text is kept only if the page confirms it, and every description is marked as AI-made.
Compare two products	Plain-language comparison across sites, ranked by what the user said matters (price, battery, rating), including products seen earlier in the session.
Place, track or return an order	Step-by-step checkout wizard; spoken read-back of quantity, total and delivery PIN; "Place order" is pressed only after the user's confirmation; track, cancel and return with reasons continue across page loads.

Conditions supported (five, each demonstrable on its own and in combination)

Blind / low vision	Voice-first overlay that works with NVDA, ChromeVox and TalkBack; spoken page overview and "where am I"; auditory switch scanning; real headings, landmarks and captioned tables; large text up to 3×; a sentence player with pause, repeat, next section and speed control.
Unclear speech	Patient listening (2.5 s silence in speech mode); an on-device matcher using word-level edit distance ("add to card" → Add to cart); LLM intent repair; two-option "did you mean"; after two misses, phraseboard for quicker communication.
Hand-motor	Full voice control; hold Space to talk; action menu of the page's controls as 44 px+ targets; two-level single-switch scanning (zones, then items) with adaptive speed; dwell to click; one-utterance actions; no time limits.
Older users	Simplified product view built from ProductJSON (no ads, nothing hidden); checkout steps; 0.75× speech; at most 3 options per step; confirmation for every action; "explain simpler"; easy undo. Colour-vision support (swatch names) is offered manually.

Bonus criteria attempted

- **More than three conditions done well:** the four above, plus accommodation for colorblind via labeling colored choices
- **Bhashini speech-to-speech:** the user speaks in their language; ASR → masked MT → grounded answer → MT back → TTS, spoken sentence by sentence while the next sentence's audio is prefetched; replays never call TTS again.
- **Graceful failure:** misheard commands ask "did you mean" or offer chips; network or model errors give a calm spoken message with Retry; after two TTS failures the device voice takes over and says so; a backend outage never leaves a silent screen.
- **Real headings, tables, captions, adjustable text and speech speed:** all answers are structured AccessibleOutput, rendered with headings and captioned tables; text scale 1–3×; speed 0.5–2× (live, no re-synthesis).
- **Low-bandwidth use:** commands, chips, playback words and confirmations are handled on the device without a server call; UI phrases and listening chimes are bundled; lite mode uses the device voice (on-device voices on Firefox for Android) and shorter answers with at most 3 options.

Safety and privacy

The assistant cannot add to cart, only request confirmation for adding to cart. The assistant never completes payment: card, CVV, OTP and UPI PIN fields and payment frames form a **secure zone** where the mic is muted, nothing is extracted and turns stop. "Buy now" and "Pay" are never pressed by the LLM; "Place order" needs an explicit high-risk confirmation.

4 Results

All language and LLM experiments ran against the live services (AnuvaadHub, Groq, Gemini); page tests use saved real Amazon.in and Flipkart pages (logged out, personal data removed) and a mock store for checkout and returns, since real orders are never placed.

Area	Result
Live use, real sites	Voice shopping on live Amazon.in works end to end. On Flipkart, controls built from React Native Web `<div>`s were invisible to every voice path; after adding them to the page map (opt-in per site; Amazon's page maps byte-identical before and after), Add to cart, Specifications and other text controls respond to voice; icon-only controls are still being covered.
Speech recognition (real voices)	9 clips, mean WER: English 8%, Hindi 0%, Telugu 19% (loanword spellings only); every spoken budget became exactly 20000; 0.5–0.7 s per clip.
Translation masking	7 placeholder styles × 3 directions: only one survived all; unmasked MT changed "Bluetooth 5.3" to "5" and transliterated brands; masked, every name, unit and price came back exact.
Extraction	Page type and schema-valid output on 10/10 saved pages; Amazon products with all 12 core fields, 33–58 specs with parsed units; 22–56 search results per page in 40–240 ms. The LLM fallback once took a bank-offer price as the price, so it never returns prices.
Grounded summaries	On 5 real product pages: 0 unsupported claims after verification; invented specs and wrong-sentiment pros are dropped; 2–4 s per product, precomputed in parallel.
Intent and safety (/turn)	14 live utterances (English, Hinglish, unclear speech, noise) handled correctly in 1–1.7 s: "isse cart mein daal do" → add-to-cart confirmation; "buy it now" → refused; confidence 0.41 → "did you mean"; noise → chips.
On-device matcher	Word-level matching: "add to card" → Add to cart (0.88, acts locally); garbled words ("paws", "nex") go to the server instead of acting wrongly, which a fuzzy-search library did not manage.
Persona runs (end to end)	6 of 6 complete on the mock store with the real backend: blind + tremor by one switch, unclear speech by keyboard, speech + motor by chips, older blind user in simple mode, older user placing an order (read-back of total and PIN), secure zone at payment.
Accessibility audit	Real Chrome, axe-core with all rules: 0 violations in 9 overlay states × 4 themes (light, dark, both with forced colours); every control ≥ 44×44 px; logical Tab order.
Automated tests	545 extension and 581 backend tests pass (3 live-API tests skipped without keys); 18 end-to-end runs (personas, A+B+C integration, Flipkart); all fixtures validated against the schemas on every pull request.

Latency. Live TTS takes 3–19 s per sentence and a cold model 6–10 s, so we warm every model at start-up, prefetch the next sentence while one plays, bundle UI phrases, and fall back to the device voice. A /turn answers in 1–2 s; chips, taps and confirmations need no LLM call.

Limitations and next steps. Checkout and order pages on real sites are tested on synthetic layouts and the mock store only (we do not place real orders); screen-reader listen-throughs on NVDA and TalkBack and calibration of condition inference on recorded sessions are still to do; Flipkart icon-only controls without text are not yet named. Next: personal voice enrollment for dysarthric speech, a TORGO-based command-accuracy evaluation, and more Bhashini languages.

Links. Working prototype: [prototype link] Demo video: [demo video link] Setup and run instructions: `README.md` in the repository.